

COMP1009

Programming Paradigms

Lecture OO-4

Aggregation

Some initial comments and reminders

- The assessment sheet is on the moodle page
 - Summarises all assessments and deadlines
- Ignore any 'no submission' emails that you had from the placeholder moodle submission.
- I have tried to add 'moodle calendar' entries for the CW deadlines instead of the moodle submission
- Are these as useful for reminding you of deadlines as the submission is? (Do they satisfy the 'need to know'.)
- CW2a deadline was last Friday
 - You may submit late submissions of up to 5 days (-5% per day)
 - I will use the latest submission to GIT at the end of the 5 days for marking and penalty calculations
 - I will add penalties later because I don't know who has ECs at the moment – so will upload unpenalised marks

The story so far...

- C code can be converted into Java code
 - Not real object oriented code though
- In Object Oriented Programming we decompose the program into a set of interacting objects
 - We identified some simple classes
 - I provided some (very) simple Java classes, PGPFFile and PGPTText to use in the program – then it was simple
 - Do Lab 3 to get your head around using these classes
- **Classes** define the structure of objects of that class
- In Java we create **objects** using the 'new' operator
- We store a reference to the resulting object in a variable of the appropriate object reference type
 - We use the . operator to apply a method to an object
- **Objects are automatically destroyed when no reference to them exists**

Some synonyms – for reference

- Clarifications because I may use the terms interchangeably without realising, and I don't want anyone to get confused
- **Methods / member functions:** functions which are defined within a class, which act upon objects of that class
 - **Class methods:** **static** function in a class – acts on the class but not on a specific object (has no 'this' object reference – see later)
 - **Instance methods:** **non-static** function – acts on a specific object
- **Attribute / data member / member variable**
 - Data that is a member of the class so that all objects get a copy of the data (assume this is not static, unless I say static member)
- **Object reference:** variable type, a reference to an object
 - Acts like a pointer to object, but uses the class name, not classname *
 - E.g. **String s = new String();** s is an object reference
 - Aside: I often call them 'object pointers' without thinking because they act exactly like restricted pointers to structs/classes in C/C++

This Lecture

- Some other class/object matters
 - Constructors
 - Static members, 'this' object reference
- Using Generics and ArrayList
- PGPText in more detail
- Aggregation & Composition
- **Aggregation is a key idea for class reuse**

Each Object contains data and/or methods

```
public class MyClass  
{  
    protected String myText;
```

Private data – label it private or protected
(more later on what these do)

```
}
```

'Getters and setters'

```
public class MyClass
{
    protected String myText;

    public void setMyText(String str)
    {
        myText = str;
    }

    public String getMyText()
    {
        return myText;
    }
}
```

Private data – label it private or protected
(more later on what these do)

'Setter' : A method which
allows us to set the value of
the data in the object

'Getter' : A method which
allows us to get the current
value of the data in the object

Main() creates an object and uses it

```
public class MyClass  
{
```

```
    protected String myText;
```

Private data – label it private or protected
(more later on what these do)

```
    public void setMyText(String str)  
    {  
        myText = str;  
    }
```

‘Setter’ : A method which
allows us to set the value of
the data in the object

```
    public String getMyText()  
    {  
        return myText;  
    }
```

‘Getter’ : A method which
allows us to get the current
value of the data in the object

```
public static void main(String[] args)  
{  
    MyClass object = new MyClass();  
    object.setMyText("Test");  
    System.out.println(object.getMyText());  
}  
}
```

Static function (main)
Creates a new object
Uses the methods on the
object

Note: We could put the main()
in any class!

New things: constructors and 'this'

We need to understand some more details now:

1. How can we initialise objects automatically, without an extra function call, avoiding:

```
MyClass object = new MyClass();  
object.setMyText("Test");
```

2. How can we access the **current** object from a method acting on that object

Constructors

Methods called automatically
to initialise new objects
You do not call them yourself!

Constructors – object initialisation

- It is often useful to be able to run some code whenever an object is created, e.g. to initialise some member variables/data
- Create a constructor to do this
 - A function which will automatically be called when object is created
- This is a function in the class which:
 1. Has the same name as the class
 2. Has no return type (not void!)
 3. Is NOT static
- e.g. for the class called **MyClass**:
// Constructor - no return type! Same name as class.
public MyClass()
{
myText = "Some text";
}
- This is a 'default constructor': no parameters needed
- In Java we can have multiple functions with the same name, as long as they have different parameter types: identify a function by name + param types₁₁

Constructors : called on creation

- Default constructor is used when you pass *no parameters* to new
 - E.g.: `MyClass object = new MyClass();`
- You can also pass parameters to constructors, e.g.:
`MyClass object = new MyClass("Hello");`
As long as you have a constructor which expects those parameters, e.g.:

```
public MyClass( String message )  
{  
    myText = message;  
}
```
- If there is no constructor which matches your parameters, the code will not compile
 - Except: if you don't create **any** constructors, a default one is created in the background (silently) which does nothing, so you can still create objects
- Important: You are not calling the constructor yourself!
 - You pass parameters to new and it then uses them to choose and call a constructor for you (it looks a bit like a call to a constructor though)

this

this ('current' object reference)

- Instance methods (non-static member functions) know which object they are acting upon

```
public MyClass( String s)
{
    myText = "text"; // myText of 'current' object
}
```

- '**this**' is an object reference to the current object
 - Often you can miss *this* out and it will be assumed

```
public MyClass( String s)
{
    this.myText = "text"; // could miss out 'this'
}
```

- Sometimes you need to tell something else which object you are working on, and you use 'this' to do so
 - You can pass 'this' as an object reference to another function

Static members

Static members (no object)

- Static members (functions and data) are **not** associated with any specific object
 - static methods have no 'this' – no current object
- You can think of them as shared between all of the objects of that class, rather than a specific object
 - `main()` is a common example
- Normally you need to specify an object for a method/function to act upon:

```
MyClass object = new MyClass();  
object.setMyText("Test");
```
- With static functions you specify a *Class* instead of *object*
 - See PGPIInput class in lab 3 for an example: `PGPIInput.get...`

Example – creating two objects

```
public class MyClass
{
    public static void main(String[] args)
    {
        MyClass object1 = new MyClass();
        MyClass object2 = new MyClass();
    }
}
```

```
public MyClass()
{
}
}
```

Constructor

Example – adding static data

```
public class MyClass
{
    public static void main(String[] args)
    {
        MyClass object1 = new MyClass();
        MyClass object2 = new MyClass();
    }
}
```

```
static int count = 0;
```

Static count is shared between all objects

```
public MyClass()
```

Constructor, uses the static data

```
{
    count++; // Uses the static member
}
```

```
}
```

Example – using static data

```
public class MyClass
{
    public static void main(String[] args)
    {
        count = 1; // set count
        MyClass.count = 0; // Set count
        MyClass object1 = new MyClass();
        MyClass object2 = new MyClass();

    }
```

Main() uses value of static count

```
static int count = 0;
```

Static count is shared between all objects

```
public MyClass()
{
    count++; // Uses the static member
}
}
```

Example – added getCount()

```
public class MyClass
{
    public static void main(String[] args)
    {
        count = 1; // set count
        MyClass.count = 0; // Set count
        MyClass object1 = new MyClass();
        MyClass object2 = new MyClass();
        System.out.println("" + getCount() + " objects created");
    }

    static int count = 0;

    public static int getCount()
    { return count; }

    public MyClass()
    {
        count++; // Uses the static member
    }
}
```



Example – full code with MyClass.

```
public class MyClass
{
    public static void main(String[] args)
    {
        count = 1; // set count
        MyClass.count = 0; // Set count
        MyClass object1 = new MyClass();
        MyClass object2 = new MyClass();
        System.out.println("" + getCount() + " objects created");
    }

    static int count = 0;

    public static int getCount()
    { return count; }

    public MyClass()
    {
        MyClass.count++; // Uses the static member
    }
}
```

main() uses value of static count

Static count is shared between all

Static function getCount() : no object

Constructor uses the static in MyClass

Note: simplified example and not thread-safe

Packages

Note: Imports and packages

- Java code is organised into a hierarchical structure of packages
 - The directory structure must match the package naming structure
 - But there can be multiple ‘base directories’ or ‘libraries’ to look in
 - Eclipse: specify package when creating class (and note ‘refactor’ option).
- Package is specified in the .java file. E.g. :

```
package lecture4;

public class MyClass
{
    public static void main(String[] args)
    {
        MyClass object1 = new MyClass();
        ...
    }
    ...
}
```

‘package lecture4’ means code must be in a directory called lecture4, in the src (source code) directory

- **For the moment, ensure your own code is all in the same package to simplify matters**
 - **i.e. don’t specify a package (even though this is bad practice)**

ArrayList

The ArrayList class

- I use an **ArrayList** class in PGPText
 - Think of ArrayList as an array with extra features
 - **Key idea: abstraction** – I don't need to know **how** a class is implemented, I only need to know its *interface*
 - We can use this class without knowing how it is implemented
- ArrayList is one of the standard Java Collection classes
 - There are many of these, and they act in standard ways
 - You can download the source files for the standard classes
 - Internally ArrayList has a java [] array to store the elements
 - So ArrayList is a wrapper around a standard array
 - Grows the array automatically as needed
 - We don't need to know its implementation to use it though!

Imports and packages

- All standard classes are in packages
 - To use a class you need to say which one (which package?)
 - The same class name could appear in multiple packages
- ArrayList is in a package called **java.util**
 - i.e. The 'util' package inside the 'java' standard package
 - So it's name is **java.util.ArrayList**
 - E.g. **java.util.ArrayList<String> array**
= new java.util.ArrayList<String>();
- We can tell the compiler what packages the classes which we use are in, to avoid repetition
 - E.g. **"import java.util.ArrayList;"**
"I want to use the ArrayList class from the java.util package"
"If I say "ArrayList", I mean the one in java.util"
 - Eclipse: use ctrl-shift-o to (try to) import standard classes

Introduction to Generic Types

- **ArrayList** is also a **generic** type:
a type which is parameterised with another class
 - Called ‘generics’ in Java, or templates in C++
 - An example of ‘parametric polymorphism’
(the usual ‘polymorphism’ in functional programming)
 - In Java a generic has angled brackets **<type>** added at the end of the name to say what type it works on
 - E.g. `<String>` means it contains Strings
 - Type “`ArrayList<String>`” means that it is an `ArrayList` with *methods* that expect and return Strings
- We will see how to create our own generic types later
 - For now we will **use** these classes rather than **creating** them
- Key factors to remember for Java generics:
 - Behaviour must be same for all parameterised types
 - Parameterised type must be a class (not a basic data type)

Important ArrayList class members

- I will use a number of methods of ArrayList:
 - Constructor - called implicitly when I create object
 - size() – tells me how many strings it contains
 - add(ob) – adds a new object to the end of the array
 - get(index) – retrieve an element from an index
 - set(index, ob) – store object ob at that index
- There are many other methods available
 - E.g. remove(), removelf(), sort(), iterator()
 - And standard operations such as reverse()
- We can also use 'for each' loops (more later)

Example code using ArrayList

```
import java.util.ArrayList;

public class MyArrayTest
{
    public static void main(String[] args)
    {
        ArrayList<String> array = new ArrayList<String>();
        array.add("Hello");
        array.add("Goodbye");
        array.add("Zoo is a later word");
        array.add("Apple is an earlier word");
        // array.add( 1 );      // Will not compile
        // array.add( 4.3f );   // Will not compile

        array.sort(null); // null means default sort

        for ( String s : array ) // For each String in array
            System.out.println(s);
    }
}
```

Abstraction, PGPText and Aggregation/Composition

Abstraction of PGPText

```
public class PGPText
```

```
{
```

```
// Constructor
```

```
public PGPText()
```

```
// Add a new line to the text storage
```

```
public void addLine( String strText )
```

```
// Retrieve a specific line from the store
```

```
public String getLine( int iLine )
```

```
// Store a specific line in the store
```

```
public void setLine( int iLine, String strText )
```

```
// Get a count of the number of lines in the store
```

```
public int getLineCount()
```

```
}
```

Abstraction is a key principle of OO design
Do not need to know the implementation of a class to use it

- We can change the **implementation** of a class later if we wish
- Key for decomposition into independent interacting classes

This is the **entire** PGPText class

```
import java.util.ArrayList;

public class PGPText
{
    // A standard collection class
    protected ArrayList<String>
        internalArray;

    public PGPText()
    {
        internalArray =
            new ArrayList<String>();
    }

    public int getLineCount()
    {
        return internalArray.size();
    }
```

Constructor
Called when
object created

Getter/accessor

Setter/mutator

```
public void addLine( String str )
{ internalArray.add(str); }

public String getLine( int iLine )
{
    return internalArray.get(iLine);
}

public void setLine(
    int iLine, String strText )
{
    while ( iLine >=
        internalArray.size() )
        internalArray.add("");
    internalArray.set(iLine, strText);
}
}
```

Setter/mutator

Key point: Internally it has an ArrayList object

```
import java.util.ArrayList;

public class PGPText
{
    // Standard collection class
    protected ArrayList<String>
        internalArray;

    public PGPText()
    { // Create internal ArrayList
        internalArray =
            new ArrayList<String>();
    }

    public int getLineCount()
    { // Get size of internal array
        return internalArray.size();
    }
}
```

Protected Data: hidden inside class

```
public void addLine( String str )
{ internalArray.add(str); }

public String getLine( int iLine )
{ return internalArray.get(iLine); }

public void setLine(
    int iLine, String strText )
{
    Grow to ensure the line to 'set' exists
    while ( iLine >=
        internalArray.size() )
        internalArray.add("");
    internalArray.set(iLine, strText);
}
}
```

Composition/Aggregation : “has-a”

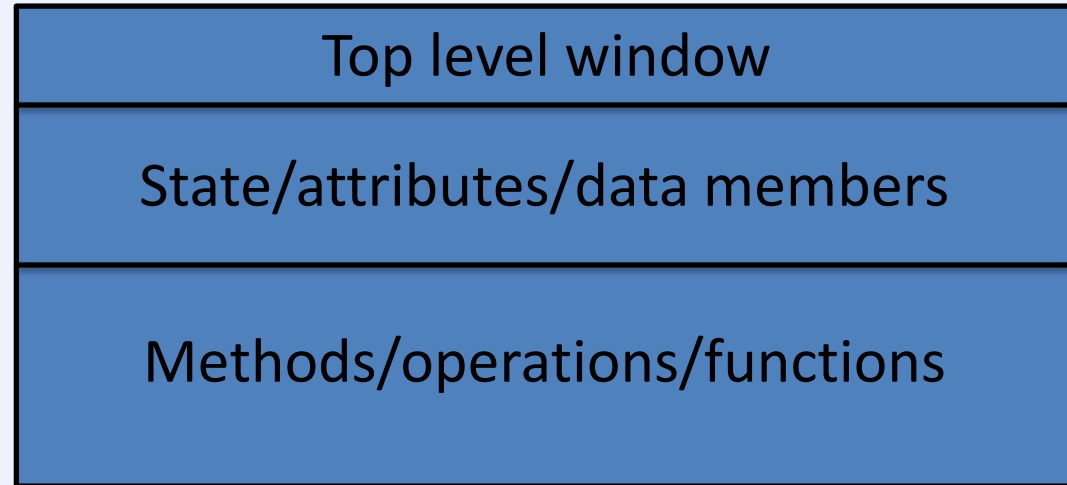
- The PGPText class ‘wraps up’ an ArrayList
 - An example of composition/aggregation
- Composition/Aggregation can be thought of as one object containing another, or ‘having’ another (HAS-A relationship)
 - Aggregation: the contents can exist on its own
 - Composition: the contents delete when the container does
- Can consider each PGPText objects as **containing** an ArrayList object (HAS-An ArrayList)
 - Hidden/protected by labelling it protected
- Constructor for my object creates the ArrayList object
 - Constructor allows me to do some initialisation code
- My object has the only object reference to it
 - Composition: ArrayList will delete when my object does
- Most PGPText methods do very little
 - They just ask the contained ArrayList to do the work (delegation)

Next Lecture

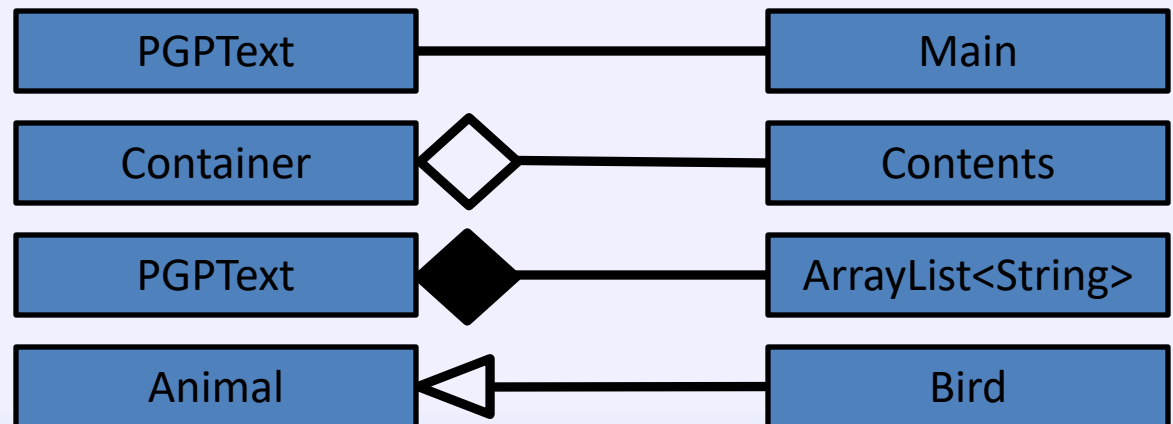
- Inheritance / sub-classes

Class diagrams - components

- Box for each class
 - Three sections:
 - Class name
 - Member data
 - Methods/functions



- Line for each relationship – symbol-end matters!
 - Association
 - Aggregation
 - Composition
 - Is-A / type of



Reminder: the PGPText class

```
import java.util.ArrayList;

public class PGPText
{
    public PGPText()
    {
        internalArray =
            new ArrayList<String>();
    }

    public int getLineCount()
    {
        return internalArray.size();
    }

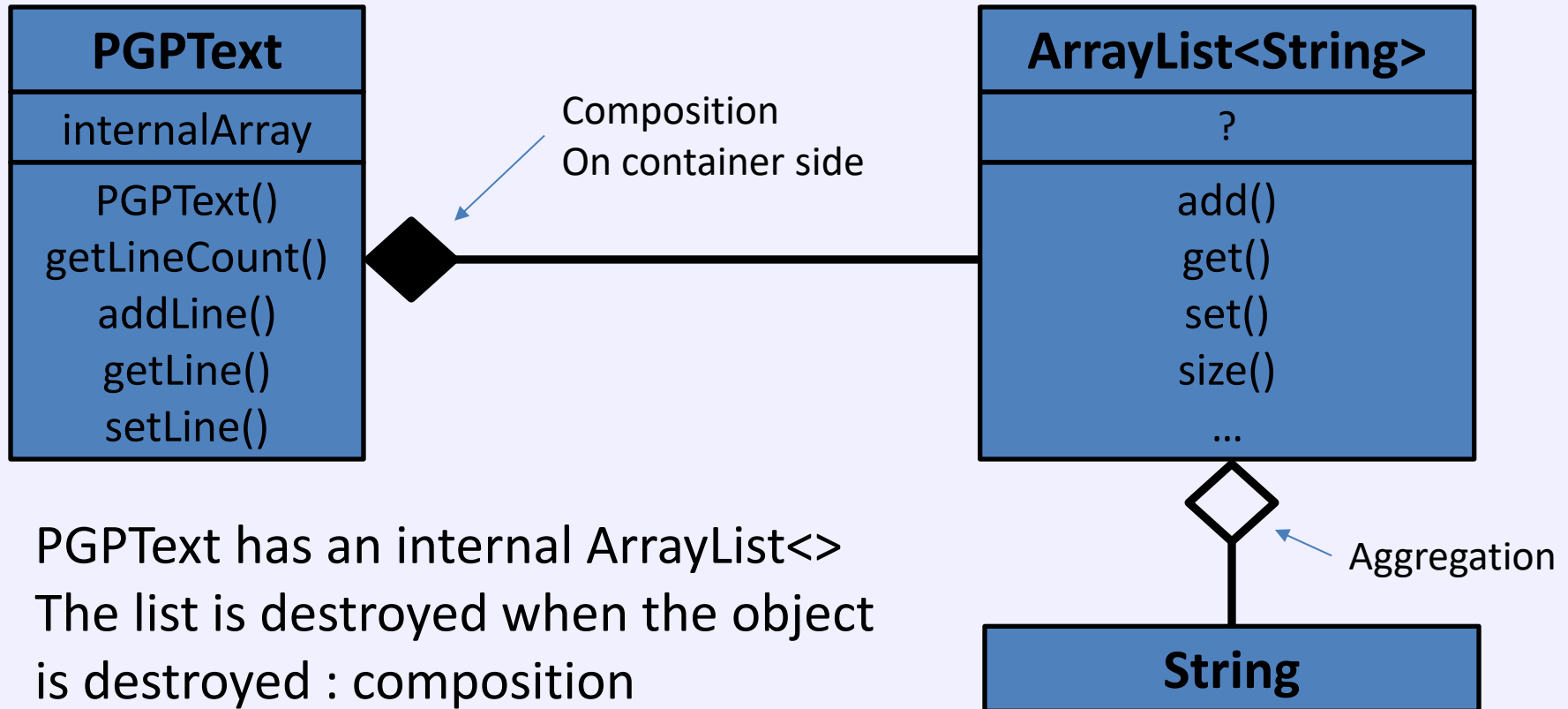
    // A standard collection class
    protected ArrayList<String>
        internalArray;
```

```
    public void addLine( String str )
    { internalArray.add(str); }

    public String getLine( int iLine )
    {
        return internalArray.get(iLine);
    }

    public void setLine(
        int iLine, String strText )
    {
        while ( iLine >=
            internalArray.size() )
            internalArray.add("");
        internalArray.set(iLine, strText);
    }
}
```

Class diagram for PGPText



- PGPText has an internal ArrayList<>
- The list is destroyed when the object is destroyed : composition
- The ArrayList<> contains Strings
- These Strings exist outside of the ArrayList<> but references to them are added to the array: aggregation

Sometimes we omit the attributes and methods
Especially for standard classes